다항식 약속 스킴 (PCS) 구현 분석

개요

Stwo 프로젝트의 다항식 약속 스킴(Polynomial Commitment Scheme, PCS)은 Circle STARK 증명 시스템의 핵심 구성 요소로, FRI(Fast Reed-Solomon Interactive Oracle Proof) 프로토콜을 Circle 도메인에 적응시켜 구현되었습니다. 이 문서는 FRI 프로토콜의 Circle 적응 방식과 Merkle 트리 기반 약속 구조를 상세히 분석합니다.

5.1 FRI 프로토콜의 Circle 적응 분석

FRI 프로토콜 기본 원리

FRI(Fast Reed-Solomon Interactive Oracle Proof)는 다항식이 특정 차수 이하임을 증명하는 대화형 증명 프로토콜입니다. Stwo에서는 이를 Circle 도메인에 적응시켜 Circle STARK의 효율성을 극대화했습니다.

핵심 구성 요소

```
// FRI 설정 구조체
pub struct FriConfig {
pub log_blowup_factor: u32, // 블로우업 팩터 (보안성 매개변수)
pub log_last_layer_degree_bound: u32, // 마지막 레이어 차수 한계
pub n_queries: usize, // 쿼리 개수 (보안성 매개변수)
}
```

블로우업 팩터 (Blowup Factor):

- 다항식의 실제 차수와 평가 도메인 크기의 비율
- 보안성과 직접적으로 연관: `security_bits = log_blowup_factor * n_queries`
- 1부터 16까지의 값을 지원하며, 값이 클수록 더 높은 보안성 제공

쿼리 개수 (n_queries):

- 검증자가 무작위로 선택하는 검증 지점의 개수
- 보안성 매개변수로, 더 많은 쿼리는 더 높은 보안성을 의미
- 증명 크기와 검증 시간에 직접적 영향

Circle 도메인에서의 FRI 적응

Circle 다항식의 Line 다항식으로의 폴딩

Stwo의 핵심 혁신은 Circle 다항식을 univariate(일변수) Line 다항식으로 효율적으로 폴딩하는 것입니다:

```
/// Circle 다항식을 Line 다항식으로 폴딩하고 누적
pub fn fold_circle_into_line(
dst: &mut [SecureField],
src: &[SecureField],
src_domain: CircleDomain,
alpha: SecureField,
) {
let alpha_sq = alpha * alpha;
src.iter()
.tuples()
.enumerate()
```

```
.for_each(|(i, (&f_p, &f_neg_p))| {
let  p = src_domain.at(bit_reverse_index(
i << CIRCLE_TO_LINE_FOLD_STEP,
src_domain.log_size(),
));
// f0(px) + py * f1(px) = 2f(p) 형태로 분해
let (mut  f0_px, mut  f1_px) = (f_p, f_neg_p);
ibutterfly(&mut  f0_px, &mut  f1_px, p.y.inverse());
let  f_prime = alpha * f1_px + f0_px;
// 누적: dst = dst * alpha^2 + f'
dst[i] = dst[i] * alpha_sq + f_prime;
});
}
```

수학적 배경:

- Circle 위의 점 `p = (px, py)` 에 대해 `2f(p) = f0(px) + py * f1(px)` 로 분해
- 폴딩된 다항식: `f' = f0 + α * f1`
- 누적 공식: `dst = dst * α² + f'`

다층 폴딩 구조

FRI 프로토콜은 다음과 같은 계층적 구조로 구현됩니다:

1. **첫 번째 레이어 (First Layer)**:

- 모든 Circle 다항식들을 커밋
- 서로 다른 크기의 다항식들을 혼합하여 처리

- Merkle 트리로 약속 생성

2. **중간 레이어들 (Inner Layers)**:

- Line 다항식들을 반복적으로 폴딩
- 각 단계에서 차수를 절반으로 감소
- 폴딩 팩터: `FOLD_STEP = 1` (매번 절반으로 감소)

3. **마지막 레이어 (Last Layer)**:

- 충분히 작은 다항식의 모든 계수를 직접 전송
- 더 이상 폴딩하지 않고 명시적으로 검증

보안성 매개변수와 효율성 트레이드오프

보안성 분석

```
impl FriConfig {
pub const fn security_bits(&self) -> u32 {
self.log_blowup_factor * self.n_queries as u32
}
}
```

보안성 계산:

- 총 보안 비트 = `log_blowup_factor × n_queries`
- 예: 블로우업 팩터 4 (log=2), 쿼리 64개 → 128비트 보안성

효율성 고려사항

증명 크기:

- 쿼리 개수에 비례하여 증가

- 블로우업 팩터가 클수록 각 레이어의 크기 증가
- 마지막 레이어 크기: `2^(log_last_layer_degree_bound + log_blowup_factor)`

검증 시간:

- 쿼리 개수에 선형 비례
- Merkle 트리 검증: `O(log(domain_size) × n_queries)`
- 폴딩 검증: 각 레이어마다 상수 시간

메모리 사용량:

- 증명자: 모든 중간 레이어를 메모리에 저장
- 검증자: 쿼리된 값들만 저장하여 메모리 효율적

Circle-to-Line 폴딩의 혁신성

기존 FRI와의 차이점

전통적인 FRI:

- 일변수 다항식에서 시작
- Reed-Solomon 코드 기반
- 이진 필드에서 주로 사용

Circle FRI:

- Circle 다항식에서 시작하여 Line 다항식으로 변환
- 복소수 원 위의 기하학적 구조 활용
- M31 필드에서의 효율적인 연산

성능 최적화

```
// 배치 역원 계산으로 나눗셈 최적화
```

```rust
fn denominator_inverses(
sample_batches: &[ColumnSampleBatch],
domain_point: CirclePoint<M31>,
) -> Vec<CM31> {
let mut denominators = Vec::new();
for sample_batch in sample_batches {
let prx = sample_batch.point.x.0;
let pry = sample_batch.point.y.0;
let pix = sample_batch.point.x.1;
let piy = sample_batch.point.y.1;
// 분모: (Pr - P).x * Pi.y - (Pr - P).y * Pi.x
denominators.push(
(prx - domain_point.x) * piy - (pry - domain_point.y) * pix
);
}
CM31::batch_inverse(&denominators) // 배치 역원 계산
}
```

최적화 기법:

- 배치 역원 계산으로 나눗셈 연산 최적화
- 비트 역순 인덱싱으로 메모리 접근 패턴 최적화
- SIMD 명령어를 활용한 병렬 처리

구현 세부사항

검증자 구조

```rust
pub struct FriVerifier<MC: MerkleChannel> {
config: FriConfig,
first_layer: FriFirstLayerVerifier<MC::H>,
inner_layers: Vec<FriInnerLayerVerifier<MC::H>>,
last_layer_domain: LineDomain,
last_layer_poly: LinePoly,
queries: Option<Queries>,
}
```

증명자 구조

```rust
pub struct FriProver<'a, B: FriOps + MerkleOps<MC::H>, MC: MerkleChannel> {
config: FriConfig,
first_layer: FriFirstLayerProver<'a, B, MC::H>,
inner_layers: Vec<FriInnerLayerProver<B, MC::H>>,
last_layer_poly: LinePoly,
}
```

핵심 특징:

- 제네릭 백엔드 지원 (`B: FriOps`)
- Merkle 채널 추상화 (`MC: MerkleChannel`)
- 생명주기 관리로 메모리 효율성 확보

에러 처리 및 견고성

```
#[derive(Clone, Copy, Debug, Error)]
pub  enum FriVerificationError {
#[error("proof contains an invalid number of FRI layers")]
InvalidNumFriLayers,
#[error("evaluations are invalid in the first layer")]
FirstLayerEvaluationsInvalid,
#[error("queries do not resolve to their commitment in the first layer")]
FirstLayerCommitmentInvalid { error: MerkleVerificationError },
#[error("degree of last layer is invalid")]
LastLayerDegreeInvalid,
#[error("evaluations in the last layer are invalid")]
LastLayerEvaluationsInvalid,
}
```

견고성 보장:

- 각 단계별 상세한 에러 타입 정의
- 중간 상태 검증으로 조기 실패 감지
- 메모리 안전성을 위한 Rust의 소유권 시스템 활용

이러한 Circle FRI 구현은 전통적인 FRI 프로토콜의 보안성을 유지하면서도 Circle STARK의 기하학적 특성을 최대한 활용하여 효율성을 크게 향상시켰습니다.

5.2 Merkle 트리 기반 약속 구조 분석

Merkle 트리 구조 개요

Stwo의 다항식 약속 스킴은 Merkle 트리를 사용하여 다항식 평가값들에 대한 암호학적 약속을 생성합니다. 이 구조는 효율적인 배치 증명과 검증을 지원하며, 메모리 사용량과 증명 크기를 최적화합니다.

기본 Merkle 트리 구조

```
pub struct MerkleProver<B: MerkleOps<H>, H: MerkleHasher> {
/// Merkle 트리의 레이어들
/// 첫 번째 레이어는 루트 레이어
/// 마지막 레이어는 가장 큰 레이어 (리프 레이어)
pub layers: Vec<Col<B, H::Hash>>,
}
```

계층적 구조:

- **루트 레이어**: 단일 해시값 (트리의 루트)
- **중간 레이어들**: 각각 이전 레이어의 2배 크기
- **리프 레이어**: 실제 다항식 평가값들을 포함

다항식 약속을 위한 Merkle 트리 구조

혼합 크기 컬럼 지원

Stwo의 혁신적인 특징 중 하나는 서로 다른 크기의 다항식들을 단일 Merkle 트리에서 효율적으로 처리하는 것입니다:

```
pub fn commit(columns: Vec<&Col<B, BaseField>>) -> Self {
// 컬럼들을 크기 순으로 정렬 (큰 것부터)
let columns = &mut columns
.into_iter()
.sorted_by_key(|c| Reverse(c.len()))
.peekable();

let mut layers: Vec<Col<B, H::Hash>> = Vec::new();
let max_log_size = columns.peek().unwrap().len().ilog2();
for log_size in (0..=max_log_size).rev() {
// 현재 log_size에 해당하는 컬럼들 수집
let layer_columns = columns
.peek_take_while(|column| column.len().ilog2() == log_size)
.collect_vec();

layers.push(B::commit_on_layer(log_size, layers.last(), &layer_columns));
}
layers.reverse();
Self { layers }
}
```

핵심 특징:

- 서로 다른 크기의 다항식들을 동시에 커밋

- 각 레이어에서 해당 크기의 컬럼들만 처리
- 메모리 효율적인 계층적 구조

노드 해싱 방식

```
impl MerkleHasher for Blake2sMerkleHasher {
fn hash_node(
children_hashes: Option<(Self::Hash, Self::Hash)>,
column_values: &[BaseField],
) -> Self::Hash {
let mut hasher = Blake2s256::new();

// 자식 노드 해시들 (중간 레이어의 경우)
if let Some((left_child, right_child)) = children_hashes {
hasher.update(left_child);
hasher.update(right_child);
}

// 컬럼 값들 (모든 레이어)
for value in column_values {
hasher.update(value.0.to_le_bytes());
}

Blake2sHash(hasher.finalize().into())
```

```
}

}
```

해싱 구조:

- **리프 노드**: 컬럼 값들만 해싱
- **중간 노드**: 자식 해시들 + 해당 레이어의 컬럼 값들
- **Blake2s**: 빠르고 안전한 해시 함수 사용

배치 증명과 검증 최적화 기법

효율적인 디커밋 프로세스

```
pub fn decommit(
&self,
queries_per_log_size: &BTreeMap<u32, Vec<usize>>,
columns: Vec<&Col<B, BaseField>>,
) -> (Vec<BaseField>, MerkleDecommitment<H>) {
let mut queried_values = vec![];
let mut decommitment = MerkleDecommitment::empty();


// 컬럼들을 레이어별로 정렬
let mut columns_by_layer = columns
.iter()
.sorted_by_key(|c| Reverse(c.len()))
.peekable();
```

```
let mut last_layer_queries = vec![];
for layer_log_size in (0..self.layers.len() as u32).rev() {
// 현재 레이어의 쿼리들 처리
let mut layer_total_queries = vec![];
// 이전 레이어 쿼리와 현재 레이어 컬럼 쿼리 병합
let mut prev_layer_queries = last_layer_queries.into_iter().peekable();
let mut layer_column_queries =
option_flatten_peekable(queries_per_log_size.get(&layer_log_size));


while let Some(node_index) =
next_decommitment_node(&mut prev_layer_queries, &mut layer_column_queries) {
// 필요한 해시 증인들 수집
if let Some(previous_layer_hashes) = previous_layer_hashes {
// 왼쪽 자식이 계산되지 않았으면 증인에 추가
if prev_layer_queries.next_if_eq(&(2 * node_index)).is_none() {
decommitment.hash_witness.push(
previous_layer_hashes.at(2 * node_index)
);
}
// 오른쪽 자식이 계산되지 않았으면 증인에 추가
if prev_layer_queries.next_if_eq(&(2 * node_index + 1)).is_none() {
decommitment.hash_witness.push(
```

```
previous_layer_hashes.at(2 * node_index + 1)
);
}
}

// 컬럼 값들 처리
let  node_values = layer_columns.iter().map(|c| c.at(node_index));
if  layer_column_queries.next_if_eq(&node_index).is_some() {
queried_values.extend(node_values); // 쿼리된 값들
} else {
decommitment.column_witness.extend(node_values); // 증인 값들
}

layer_total_queries.push(node_index);
}

last_layer_queries = layer_total_queries;
}

(queried_values, decommitment)
}
```

배치 증명의 핵심 최적화

1. 쿼리 병합 (Query Merging):

- 여러 레이어의 쿼리들을 효율적으로 병합
- 중복 계산 제거로 증명 크기 감소
- `next_decommitment_node` 함수로 최적 순서 결정

2. 선택적 증인 생성:

- 검증자가 계산할 수 있는 값들은 제외
- 필요한 해시와 컬럼 값들만 증인에 포함
- 증명 크기 최소화

3. 계층적 처리:

- 각 레이어를 독립적으로 처리
- 메모리 사용량 최적화
- 병렬 처리 가능한 구조

메모리 효율성과 증명 크기 최적화

메모리 최적화 전략

1. 지연 계산 (Lazy Computation):

```
// 필요한 시점에만 해시 계산
let layer_columns = columns_by_layer
.peek_take_while(|column| column.len().ilog2() == layer_log_size)
.collect_vec();
```

2. 스트리밍 처리:

- 전체 트리를 메모리에 유지하지 않음

- 레이어별 순차 처리로 메모리 사용량 제한
- 큰 다항식도 효율적으로 처리 가능

3. 비트 역순 인덱싱:

```
// 메모리 접근 패턴 최적화
let domain_point = commitment_domain.at(
bit_reverse_index(query_position, log_size)
);
```

증명 크기 최적화

1. 최소 증인 생성:

```
pub struct MerkleDecommitment<H: MerkleHasher> {
/// 검증자가 추론할 수 없는 해시 값들만 포함
pub hash_witness: Vec<H::Hash>,
/// 검증자가 추론할 수 없는 컬럼 값들만 포함
pub column_witness: Vec<BaseField>,
}
```

2. 압축 기법:

- 중복 해시 제거
- 계산 가능한 값들 생략
- 효율적인 직렬화

3. 배치 검증:

- 여러 쿼리를 동시에 처리
- 공통 경로 공유로 증명 크기 감소
- 검증 시간 단축

검증 프로세스 최적화

효율적인 검증 알고리즘

```
pub fn verify(
&self,
queries_per_log_size: &BTreeMap<u32, Vec<usize>>,
queried_values: Vec<BaseField>,
decommitment: MerkleDecommitment<H>,
) -> Result<(), MerkleVerificationError> {
let mut queried_values = queried_values.into_iter();
let mut hash_witness = decommitment.hash_witness.into_iter();
let mut column_witness = decommitment.column_witness.into_iter();

let mut last_layer_hashes: Option<Vec<(usize, H::Hash)>> = None;
for layer_log_size in (0..=*max_log_size).rev() {
// 각 레이어별 검증 수행
// 해시 재계산 및 일치성 확인
// 다음 레이어로 전파
}
```

```
// 최종 루트 해시 검증
let [(_, computed_root)] = last_layer_hashes.unwrap().try_into().unwrap();
if  computed_root != self.root {
return Err(MerkleVerificationError::RootMismatch);
}

Ok(())
}
```

검증 최적화 기법

1. 점진적 검증:

- 각 레이어별로 순차 검증
- 조기 실패 감지로 불필요한 계산 방지
- 메모리 사용량 최소화

2. 배치 해시 계산:

- 여러 노드의 해시를 동시에 계산
- SIMD 명령어 활용 가능
- CPU 캐시 효율성 향상

3. 에러 처리 최적화:

```
#[derive(Clone, Copy, Debug, Error, PartialEq, Eq)]
pub  enum MerkleVerificationError {
#[error("Witness is too short")]
```

```
WitnessTooShort,

#[error("Witness is too long.")]

WitnessTooLong,

#[error("too many Queried values")]

TooManyQueriedValues,

#[error("too few queried values")]

TooFewQueriedValues,

#[error("Root mismatch.")]

RootMismatch,

}
```

보안성과 효율성의 균형

보안성 보장

1. 암호학적 해시 함수:

- Blake2s: 256비트 출력, 고속 처리
- 충돌 저항성과 일방향성 보장
- 양자 컴퓨터에 대한 상당한 저항성

2. 완전성과 건전성:

- 완전성: 정직한 증명자는 항상 검증 통과
- 건전성: 거짓 명제에 대한 증명은 높은 확률로 거부
- 지식 건전성: 증명자는 실제로 다항식을 알고 있어야 함

효율성 최적화

1. 시간 복잡도:

- 커밋: O(n log n) - n은 다항식 크기
- 증명 생성: O(k log n) - k는 쿼리 개수
- 검증: O(k log n)

2. 공간 복잡도:

- 증명자: O(n) - 전체 다항식 저장
- 검증자: O(k) - 쿼리된 값들만 저장
- 증명 크기: O(k log n)

3. 통신 복잡도:

- 라운드 수: 상수 (비대화형)
- 메시지 크기: O(k log n)
- 대역폭 효율적

이러한 Merkle 트리 기반 약속 구조는 Circle STARK의 효율성을 극대화하면서도 강력한 보안성을 제공하여, 실용적인 영지식 증명 시스템을 구현할 수 있게 합니다.